

SSD

Aplicación de Cabina para Radio Estudiantil

Especificación de Sistema y Estándares de Desarrollo

1. Objetivo del Sistema

Desarrollar una aplicación de cabina para radio estudiantil que permita gestionar, reproducir y controlar contenido musical local, facilitando la operación de transmisión mediante herramientas de automatización, control de dispositivos de audio y soporte para la interacción en vivo del operador.

El sistema debe permitir al operador de cabina reproducir música, gestionar playlists, monitorear pistas en preescucha y controlar el micrófono, manteniendo una operación simple y eficiente durante la transmisión.

2. Requisitos Funcionales

2.1 Gestión de Biblioteca Musical

El sistema debe permitir gestionar una biblioteca musical almacenada localmente.

Funciones:

- Reproducir música desde una biblioteca local
- Almacenar canciones en el sistema de persistencia local
- Buscar canciones dentro de la biblioteca
- Visualizar canciones disponibles
- Mantener la persistencia de las canciones almacenadas

2.2 Gestión de Playlists y Cola de Reproducción

El sistema debe permitir la administración de listas de reproducción.

Funciones:

- Mantener una cola de canciones
- Cambiar la canción actual
- Añadir canciones a la cola
- Eliminar canciones de la cola
- Guardar playlists
- Cargar playlists desde archivo (formato por definir)

2.3 Control de Reproducción

El sistema debe permitir controlar el comportamiento de la reproducción de audio.

Funciones:

- Iniciar reproducción
- Pausar reproducción
- Cambiar de canción
- Controlar el volumen de reproducción

2.4 Sistema de Doble Plato

El sistema debe implementar un modelo de doble plato de reproducción para operación en cabina.

Plato Principal	Plato Secundario
Contiene la pista que está siendo reproducida al aire .	Contiene la pista en preescucha dentro de la cabina.

Reglas:

- El segundo plato debe reproducirse en un dispositivo de preescucha
- La preescucha no debe salir al audio principal

2.5 Gestión de Dispositivos de Audio

El sistema debe detectar y permitir seleccionar dispositivos de audio disponibles.

Funciones:

- Detectar dispositivos de entrada de audio
- Detectar dispositivos de salida de audio
- Seleccionar dispositivos de entrada
- Seleccionar dispositivos de salida

Tipos de Salida:

Dispositivo Global	Dispositivo Privado
Parlantes o sistema de sonido que reproduce la señal principal .	Audífonos o monitor de cabina utilizados para la preescucha .

2.6 Grabación y Redirección de Audio

El sistema debe permitir capturar audio desde un dispositivo de entrada.

Funciones:

- Grabar audio desde el micrófono
- Redirigir la entrada de audio hacia la salida seleccionada

2.7 Modo DJ (Intervención del Operador)

El sistema debe permitir la intervención del operador mediante el micrófono.

Funciones:

- Activar micrófono
- Activar modo habla (Talking mode)
- Aplicar ducking al audio musical mientras el operador habla

■ Ducking

Reducción automática del volumen de la música cuando el micrófono está activo.

2.8 Descarga de Música

El sistema debe permitir obtener contenido musical externo.

Funciones:

- Descargar música desde una fuente externa
- Almacenar las canciones descargadas en la biblioteca local
- Registrar automáticamente las canciones en el sistema de persistencia

3. Requisitos de Interfaz

La interfaz del sistema debe cumplir las siguientes características:

- Ser simple y clara para operadores de cabina
- Permitir acceso rápido a los controles principales

Mostrar:

- Canción actual
- Cola de reproducción
- Estado del sistema
- Control de dispositivos
- Control del micrófono

4. Requisitos de Plataforma

El sistema debe ser **multiplataforma**. Plataformas soportadas:

■ Windows

■ Linux

5. Sistema de Estados

El sistema debe implementar una máquina de estados que controle el comportamiento operativo.

Off_air	El sistema está inactivo y no transmite audio.
On_air	El sistema está activo y preparado para transmitir.
Playing	Se está reproduciendo una pista en el plato principal.
Paused	La reproducción se encuentra detenida temporalmente.
Preview	Una pista se reproduce en el plato secundario para preescucha.
Talking	El operador está hablando. El sistema aplica ducking a la música.

6. Persistencia de Datos

El sistema debe almacenar información localmente:

■ Biblioteca musical

■ Playlists

■ Metadatos de canciones

■ Configuración de dispositivos

7. Consideraciones de Operación

El sistema debe:

■ Mantener la reproducción continua sin interrupciones

- Permitir cambiar canciones de forma inmediata
- Garantizar que la preescucha no interfiera con la salida principal
- Permitir control rápido del micrófono durante transmisiones en vivo

Estándares de Desarrollo

Convenciones de Nomenclatura

Convención	Uso	Ejemplo
camelCase	Variables, funciones, métodos, columnas SQL	playTrack, songId
PascalCase	Clases y archivos	AudioManager, TrackPlayer
UPPER_CASE	Constantes	MAX_VOLUME, DEFAULT_PATH

Las variables y métodos deben ser **auto descriptivos**. Las funciones y métodos deben comenzar con la acción que realizan. No se usará **snake_case**.

Ejemplos de métodos:

```
playTrack loadPlaylist getDevice  
setVolume detectAudioDevices
```

Prefijos para booleanos:

```
isPlaying hasAudioInput canPlay
```

Las variables, métodos, clases y commits deberán escribirse en **inglés**.

Base de Datos

Tablas — definirse en singular:

```
song playlist user device
```

Columnas — usar camelCase:

```
songId filePath createdAt updatedAt duration
```

Formato de Fechas y Tiempo

Tipo	Formato	Ejemplo
Fechas	YYYY-MM-DD	2026-03-07
Duraciones de audio	mm:ss	03:45

Convenciones de Commits

Los commits deberán seguir el formato:

Tipo	Uso
feat :	Nueva funcionalidad
fix :	Corrección de errores
refactor :	Mejora interna sin cambio funcional
docs :	Documentación

Reglas: inglés · máximo 72 caracteres · un cambio lógico por commit

Ejemplo:

```
feat: add audio device detection
```

Documentación del Código

No es obligatorio documentar todas las funciones, pero se recomienda usar **docstrings** o comentarios.

```
/**
 * Reproduce una pista desde la biblioteca
 */
```

Reglas para Funciones

Máximo 50 líneas

Máximo 5 parámetros

Máximo 3 niveles de indentación

Si una función supera estos límites, deberá dividirse. Cada función debe tener **una única responsabilidad**.

Manejo de Errores

Nunca ignorar errores. Todo lo que pueda generar un error de runtime deberá manejarse con **try-catch**.

No capturar excepciones genéricas cuando exista una específica:

■ **Incorrecto**

```
catch(Exception e)
```

■ **Correcto**

```
catch(AudioDeviceException e)
```

Logging

Se debe usar **logger** en lugar de **print**. Niveles disponibles:

DEBUG

INFO

WARNING

ERROR

CRITICAL

```
logger.info("Track started")
logger.error("Audio device not available")
```

Códigos de Error

Los códigos de error deberán seguir el formato **RA-###**:

RA-001 RA-002 RA-003

Cada código debe tener una explicación documentada dentro del archivo **errors.md**.

Tamaño de Archivos

Límite recomendado: 750 líneas

Si un archivo supera este límite, se deberán dividir responsabilidades en módulos separados.

Organización del Proyecto

El proyecto deberá seguir una estructura basada en **Clean Architecture**:

```
src/
  domain ← Entidades y reglas de negocio
  application ← Casos de uso
  infrastructure ← Implementaciones externas (DB, audio)
  ui ← Interfaz de usuario
  core ← Utilidades compartidas
```

Reglas de Dependencia:

■ Permitido	■ No permitido
UI → Application → Domain Infrastructure → Domain	Domain → UI

Convenciones de Clases

Las clases deben representar entidades claras y seguir el **Single Responsibility Principle**:

```
AudioManager TrackPlayer PlaylistService
DeviceRepository DownloadService
```

Importaciones

No se deben usar importaciones globales:

■ Incorrecto	■ Correcto
from module import *	from audio.deviceManager import DeviceManager

Configuración del Sistema

No se deben **hardcodear** valores de configuración. Los valores configurables deberán almacenarse en archivos de configuración (**config.json**):

```
audioBufferSize defaultVolume libraryPath
```

Comentarios de Desarrollo

Usar etiquetas estándar para marcar código pendiente:

```
TODO: implement crossfade between tracks  
FIXME: audio buffer overflow when switching devices
```

Testing

Los tests deberán ubicarse en la carpeta **tests/**:

```
AudioManagerTest PlaylistServiceTest  
DeviceManagerTest
```

Estándar de Arquitectura: Clean Architecture

- Separación clara de responsabilidades
- Independencia entre capas
- El dominio no depende de frameworks ni de la interfaz de usuario